

MCT 332/MCT334 Design of MCT 2

Spring 2026

ROS2 Simulation Setup



Lab Objectives:

- I. Integrate a URDF Robot in ROS2
 - Create a ROS2 package and import URDF files.
 - Visualize and validate the robot model in RViz.
- II. Configure ROS2 Control
 - Implement and tune a differential drive controller.
 - Publish velocity commands and verify odometry output.
- III. Simulate the Robot in Gazebo
 - Launch the robot in a custom simulation world.
- IV. Integrate and Test Sensors
 - Add IMU and LiDAR sensors in URDF.
 - Verify sensor data topics in ROS2.
- V. Implement Sensor Fusion
 - Configure EKF using robot_localization.
 - Fuse IMU and odometry for improved localization.
- VI. Develop Basic Obstacle Avoidance
 - Create a Python ROS2 node using LaserScan data.
 - Implement simple reactive navigation behavior.

Steps:

➤ *Install Needed Packages*

- `sudo apt install ros-$ROS_DISTRO-joint-state-publisher-gui`
- `sudo apt install ros-$ROS_DISTRO-xacro`
- `sudo apt install ros-$ROS_DISTRO-robot-localization`

➤ URDF Setup

- I. After the URDF exporting, we need to create ROS2 package and copy all the needed files from the URDF package. In your workspace source folder, create a ROS2 package:

```
cd ~/design_ws/src
```

```
ros2 pkg create --build-type ament_cmake mobile_robot_sim
```

- II. Copy the needed folders urdf, config and meshes folders (if there is textures in your robot you can take this folder too) and paste all these folders into your created package.

- III. We need to display the mobile robot in Rviz to make sure that all the joints rotate in the desired direction and all positioning of the robot joints are in their places.

- a. First, create folder named **rviz** to define the configuration file for rviz and create also a folder named **launch** to generate the launch file.

```
cd ~/design_ws/src/mobile_robot_sim
```

```
mkdir launch rviz
```

- b. In rviz folder, create file named **config.rviz** and paste the following:

```
cd rviz && touch config.rviz
```

```

Panels:
- Class: rviz_common/Displays
  Help Height: 78
  Name: Displays
  Property Tree Widget:
    Expanded:
      - /Global Options1
      - /Status1
      - /RobotModel1/Links1
      - /TF1
    Splitter Ratio: 0.5
  Tree Height: 557
Visualization Manager:
Class: ""
Displays:
- Alpha: 0.5
  Cell Size: 1
  Class: rviz_default_plugins/Grid
  Color: 160; 160; 164
  Enabled: true
  Name: Grid
- Alpha: 0.6
  Class: rviz_default_plugins/RobotModel
  Description Topic:
    Depth: 5
    Durability Policy: Volatile
    History Policy: Keep Last
    Reliability Policy: Reliable
    Value: /robot_description
  Enabled: true
  Name: RobotModel
  Visual Enabled: true
- Class: rviz_default_plugins/TF
  Enabled: true
  Name: TF
  Marker Scale: 0.3
  Show Arrows: true
  Show Axes: true
  Show Names: true
Enabled: true
Global Options:
  Background Color: 48; 48; 48
  Fixed Frame: RobotBody
  Frame Rate: 30
Name: root
Tools:
- Class: rviz_default_plugins/Interact
  Hide Inactive Objects: true
- Class: rviz_default_plugins/MoveCamera
- Class: rviz_default_plugins/Select
- Class: rviz_default_plugins/FocusCamera
- Class: rviz_default_plugins/Measure
  Line color: 128; 128; 0
Transformation:
  Current:
    Class: rviz_default_plugins/TF
Value: true
Views:
  Current:
    Class: rviz_default_plugins/Orbit
    Name: Current View
    Target Frame: <Fixed Frame>
    Value: Orbit (rviz)
Saved: ~

```

c. Create a launch file and paste the following:

```
cd ../launch && touch display_rviz.launch.py
```

```

from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.conditions import IfCondition, UnlessCondition
from launch.substitutions import Command, LaunchConfiguration
from launch_ros.actions import Node
from launch_ros.substitutions import FindPackageShare
import os

def generate_launch_description():
    pkg_share = FindPackageShare(package='mobile_robot_sim').find('mobile_robot_sim')
    default_model_path = os.path.join(pkg_share, 'urdf', 'Robot_Body_URDF.urdf')
    default_rviz_config_path = os.path.join(pkg_share, 'rviz', 'config.rviz')

    robot_state_publisher_node = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        parameters=[{'robot_description': Command(['xacro ',
LaunchConfiguration('model')])}]
    )
    joint_state_publisher_node = Node(
        package='joint_state_publisher',
        executable='joint_state_publisher',
        name='joint_state_publisher',
        parameters=[{'robot_description': Command(['xacro ', default_model_path])}],
        condition=UnlessCondition(LaunchConfiguration('gui'))
    )
    joint_state_publisher_gui_node = Node(
        package='joint_state_publisher_gui',
        executable='joint_state_publisher_gui',
        name='joint_state_publisher_gui',
        condition=IfCondition(LaunchConfiguration('gui'))
    )
    rviz_node = Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
        output='screen',
        arguments=['-d', LaunchConfiguration('rvizconfig')],
    )

    return LaunchDescription([
        DeclareLaunchArgument(name='gui', default_value='True', description='Flag to
enable joint_state_publisher_gui'),
        DeclareLaunchArgument(name='model', default_value=default_model_path,
description='Absolute path to robot model file'),
        DeclareLaunchArgument(name='rvizconfig', default_value=default_rviz_config_path,
description='Absolute path to rviz config file'),
        joint_state_publisher_node,
        joint_state_publisher_gui_node,
        robot_state_publisher_node,
        rviz_node
    ])

```

d. Add the folders in the share section in CMakeLists.txt

```

install(
  DIRECTORY urdf meshes launch rviz config
  DESTINATION share/${PROJECT_NAME}
)

```

e. Add the dependencies in package.xml

```

<exec_depend>joint_state_publisher</exec_depend>
<exec_depend>joint_state_publisher_gui</exec_dep
end>
<exec_depend>robot_state_publisher</exec_depend>
<exec_depend>rviz</exec_depend>
<exec_depend>xacro</exec_depend>

```

f. Build your workspace, then launch the file.

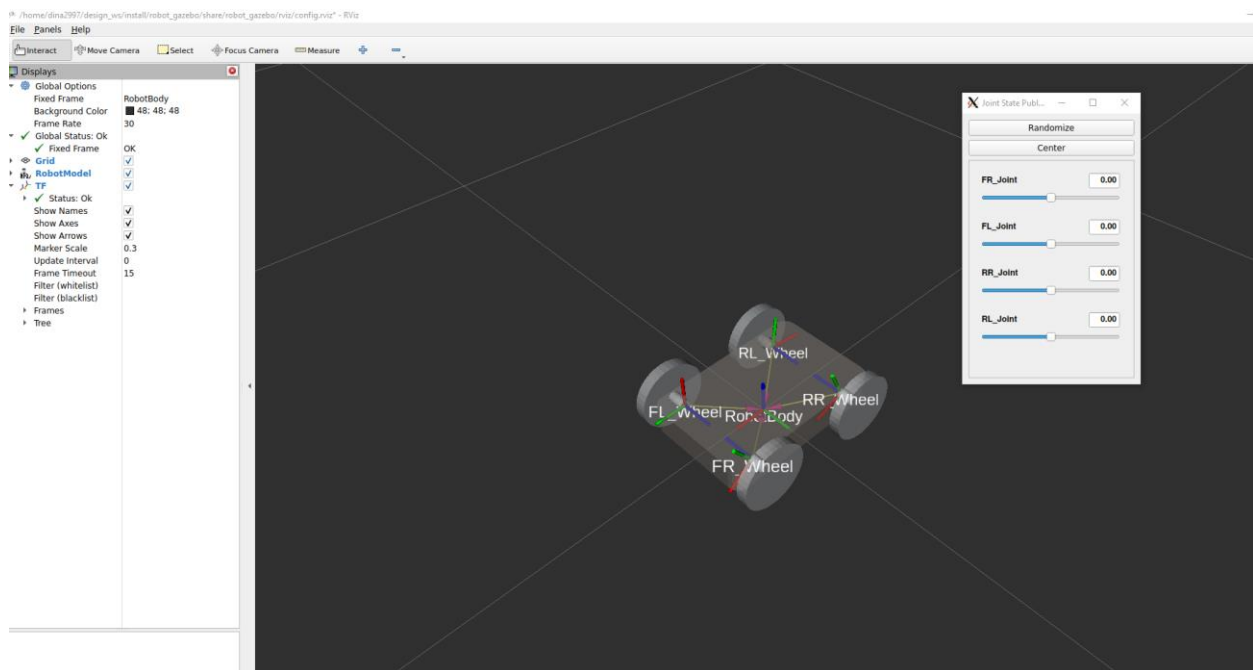
```
cd ~/design_ws
```

```
colcon build --symlink-install --packages-select  
mobile_robot_sim
```

```
source install/setup.bash
```

```
ros2 launch mobile_robot_sim display_rviz.launch.py
```

You will see as the following and there is a joint state publisher gui that you can rotate the joint through to make sure that all the joints are in the same direction for forward and backward. If you found any joint in opposite direction, just change the direction in the <axis> definition in <joint> to be negative.



➤ Joint Controllers Setup

After checking for all the setup files, we can define the controllers for the joints of the wheels.

I. In the **config** folder, create **controllers.yaml** and paste the following:

```
cd ~/design_ws/src/mobile_robot_sim/config && touch  
controllers.yaml
```

```

---
controller_manager:
  ros__parameters:
    update_rate: 50 # Hz
    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster
    diff_drive_controller:
      type: diff_drive_controller/DiffDriveController
diff_drive_controller:
  ros__parameters:
    left_wheel_names: ['FL_Joint', 'RL_Joint']
    right_wheel_names: ['FR_Joint', 'RR_Joint']

    wheel_separation: 0.22
    wheel_radius: 0.06

    wheel_separation_multiplier: 1.0
    left_wheel_radius_multiplier: 1.0
    right_wheel_radius_multiplier: 1.0

    publish_rate: 50.0
    odom_frame_id: odom
    base_frame_id: RobotBody
    pose_covariance_diagonal: [0.001, 0.001, 0.001, 0.001, 0.001, 0.01]
    twist_covariance_diagonal: [0.001, 0.001, 0.001, 0.001, 0.001, 0.01]
    open_loop: true
    enable_odom_tf: true

    cmd_vel_timeout: 0.5
    use_stamped_vel: true # for sensor fusion it must be stamped

    linear.x.has_velocity_limits: true
    linear.x.has_acceleration_limits: true
    linear.x.has_jerk_limits: false
    linear.x.max_velocity: 1.0 # m/s
    linear.x.min_velocity: -1.0
    linear.x.max_acceleration: 1.0
    linear.x.max_jerk: 0.0
    linear.x.min_jerk: 0.0

    angular.z.has_velocity_limits: true
    angular.z.has_acceleration_limits: true
    angular.z.has_jerk_limits: false
    angular.z.max_velocity: 1.0
    angular.z.min_velocity: -1.0
    angular.z.max_acceleration: 1.0
    angular.z.min_acceleration: -1.0
    angular.z.max_jerk: 0.0
    angular.z.min_jerk: 0.0

```

II. Edit the urdf file and add the following define the joint controllers for gz sim:

```
<ros2_control name="RobotSystem" type="system">
  <hardware>
    <plugin>gz_ros2_control/GazeboSimSystem</plugin>
  </hardware>
  <joint name="FR_Joint">
    <command_interface name="velocity">
      <param name="min">-10</param>
      <param name="max">10</param>
    </command_interface>
    <state_interface name="velocity"/>
    <state_interface name="position"/>
  </joint>
  <joint name="FL_Joint">
    <command_interface name="velocity">
      <param name="min">-10</param>
      <param name="max">10</param>
    </command_interface>
    <state_interface name="velocity"/>
    <state_interface name="position"/>
  </joint>
  <joint name="RL_Joint">
    <command_interface name="velocity">
      <param name="min">-10</param>
      <param name="max">10</param>
    </command_interface>
    <state_interface name="velocity"/>
    <state_interface name="position"/>
  </joint>
  <joint name="RR_Joint">
    <command_interface name="velocity">
      <param name="min">-10</param>
      <param name="max">10</param>
    </command_interface>
    <state_interface name="velocity"/>
    <state_interface name="position"/>
  </joint>
</ros2_control>

<gazebo>
  <plugin filename="gz_ros2_control-system"
    name="gz_ros2_control::GazeboSimROS2ControlPlugin">
    <parameters>
      /home/dina2997/design_ws/install/robot_gazebo/share/robot_gazebo/config/control
      lers.yaml
    </parameters>
  </plugin>
</gazebo>
```

III. Create a launch file to launch the robot with the controllers in gz sim and paste the following:

```
cd ../launch && touch robot_gazebo.launch.py
```

```

from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription, GroupAction,
RegisterEventHandler, DeclareLaunchArgument
from launch.conditions import IfCondition, UnlessCondition
from launch.event_handlers import OnProcessStart
from launch.substitutions import Command, LaunchConfiguration,
PathJoinSubstitution
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node
from launch_ros.substitutions import FindPackageShare
import os
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    pkg_share = FindPackageShare(package='robot_gazebo').find('robot_gazebo')
    default_model_path = os.path.join(pkg_share, 'urdf',
'Robot_Body_URDF.urdf')
    #default_rviz_config_path = os.path.join(pkg_share, 'rviz', 'config.rviz')
    robot_controllers = PathJoinSubstitution([pkg_share, 'config',
'controllers.yaml'])

    robot_state_publisher_node = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        parameters=[{'robot_description': Command(['xacro
',default_model_path]), 'use_sim_time': True}]
    )
    # gz launch
    gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(
            get_package_share_directory('ros_gz_sim'),
            'launch',
            'gz_sim.launch.py'
        )]),
        launch_arguments={'gz_args': '-r empty.sdf'}.items())
    #Bridge Node
    bridge_gz = Node(
        package='ros_gz_bridge',
        executable='parameter_bridge',
        arguments=[
            # General
            '/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock',

            # Gazebo_Control
            '/cmd_vel@geometry_msgs/msg/Twist[gz.msgs.Twist',
            '/odom@nav_msgs/msg/Odometry[gz.msgs.Odometry',
            '/joint_states@sensor_msgs/msg/JointState[gz.msgs.Model',
        ],
        remappings=[
            ('/world/empty/model/Robot_Body_URDF/joint_state', 'joint_states'),
        ],
        output='screen'
    )
    # gz spawn robot entity
    node_gz_spawn_entity = Node(
        package='ros_gz_sim',
        executable='create',
        output='screen',
        arguments=['-topic', 'robot_description',
            '-name', 'Robot_Body_URDF',
            '-x', '0.0',
            '-y', '0.0',
            '-z', '0.03',
            '-R', '0.0',    # rotate around X (roll)
            '-P', '0.0',
            '-Y', '0.0'
        ],
        parameters=[{"use_sim_time": True}]
    )
    # controller spawn
    joint_state_broadcaster_spawner = Node(
        package="controller_manager",
        executable="spawner",
        arguments=["joint_state_broadcaster"],

```

```

diff_drive_controller_spawner = Node(
    package="controller_manager",
    executable="spawner",
    arguments=["diff_drive_controller"
    ],
)
control_node = Node(
    package='controller_manager',
    executable='ros2_control_node',
    parameters=[{'robot_description': Command(['xacro
',default_model_path])}], robot_controllers, {"use_sim_time": True}],
    output='screen',
)

return LaunchDescription([
    gazebo,
    bridge_gz,
    node_gz_spawn_entity,
    robot_state_publisher_node,
    control_node,
    joint_state_broadcaster_spawner,
    diff_drive_controller_spawner,

])

```

IV. Add the dependencies in package.xml

```

<depend>ros2_control</depend>
<depend>ros2_controllers</depend>
<depend>ros_gz_sim</depend>
<depend>controller_manager</depend>

```

V. Build workspace and run the launch file

```

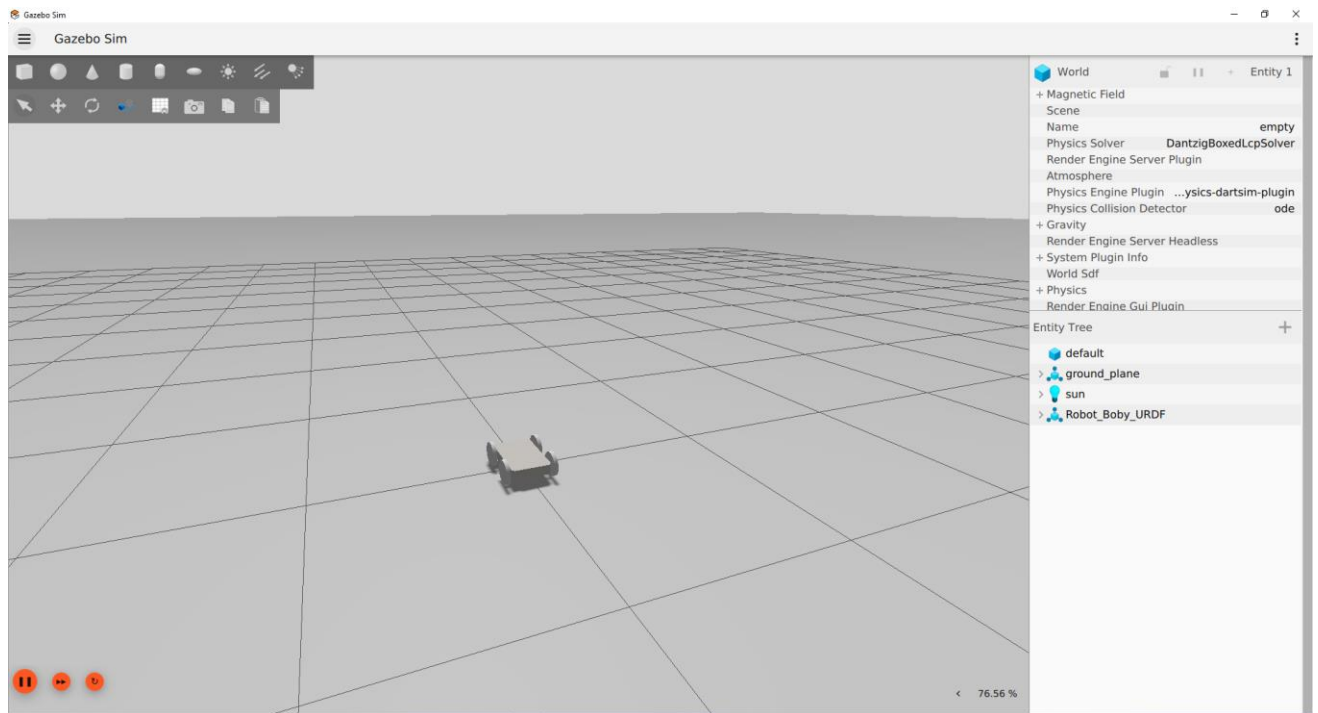
cd ~/design_ws

colcon build --symlink-install --packages-select
mobile_robot_sim

source install/setup.bash

ros2 launch mobile_robot_sim robot_gazebo.launch.py

```



➤ Environment Setup

The next step, we need to setup the environment where the mobile robot will work

I. Create a folder in the package named ***world***

```
cd ~/design_ws/src/mobile_robot_sim && mkdir world
```

II. In the world folder, create a file named ***my_world.sdf*** and paste the following:

```
cd world && touch my_world.sdf
```

```

<sdf version='1.7'>
  <world name='my_world'>
    <physics name="1ms" type="ignored">
      <max_step_size>0.001</max_step_size>
      <real_time_factor>1.0</real_time_factor>
    </physics>
    <plugin
      filename="gz-sim-physics-system"
      name="gz::sim::systems::Physics">
    </plugin>
    <plugin
      filename="gz-sim-user-commands-system"
      name="gz::sim::systems::UserCommands">
    </plugin>
    <plugin
      filename="gz-sim-scene-broadcaster-system"
      name="gz::sim::systems::SceneBroadcaster">
    </plugin>
    <plugin filename="gz-sim-imu-system"
      name="gz::sim::systems::Imu">
    </plugin>
    <plugin
      filename="gz-sim-sensors-system"
      name="gz::sim::systems::Sensors">
      <render_engine>ogre2</render_engine>
    </plugin>
    <light name='sun' type='directional'>
      <cast_shadows>1</cast_shadows>
      <pose>0 0 10 0 -0 0</pose>
      <diffuse>0.8 0.8 0.8 1</diffuse>
      <specular>0.2 0.2 0.2 1</specular>
      <attenuation>
        <range>1000</range>
        <constant>0.9</constant>
        <linear>0.01</linear>
        <quadratic>0.001</quadratic>
      </attenuation>
      <direction>-0.5 0.1 -0.9</direction>
      <spot>
        <inner_angle>0</inner_angle>
        <outer_angle>0</outer_angle>
        <falloff>0</falloff>
      </spot>
    </light>
    <model name='ground_plane'>
      <static>1</static>
      <link name='link'>
        <collision name='collision'>
          <geometry>
            <plane>
              <normal>0 0 1</normal>
              <size>100 100</size>
            </plane>
          </geometry>
        </collision>
      </link>
    </model>
  </world>
</sdf>

```

```

    <surface>
      <friction>
        <ode>
          <mu>100</mu>
          <mu2>50</mu2>
        </ode>
        <torsional>
          <ode/>
        </torsional>
      </friction>
      <contact>
        <ode/>
      </contact>
      <bounce/>
    </surface>
    <max_contacts>10</max_contacts>
  </collision>
  <visual name='visual'>
    <cast_shadows>0</cast_shadows>
    <geometry>
      <plane>
        <normal>0 0 1</normal>
        <size>100 100</size>
      </plane>
    </geometry>
    <material>
      <script>
        <uri>file://media/materials/scripts/gazebo.material</uri>
        <name>Gazebo/Grey</name>
      </script>
    </material>
  </visual>
  <self_collide>0</self_collide>
  <enable_wind>0</enable_wind>
  <kinematic>0</kinematic>
</link>
</model>
<gravity>0 0 -9.8</gravity>
<magnetic_field>6e-06 2.3e-05 -4.2e-05</magnetic_field>
<atmosphere type='adiabatic'/>
<physics type='ode'>
  <max_step_size>0.001</max_step_size>
  <real_time_factor>1</real_time_factor>
  <real_time_update_rate>1000</real_time_update_rate>
</physics>
<scene>
  <ambient>0.4 0.4 0.4 1</ambient>
  <background>0.7 0.7 0.7 1</background>
  <shadows>1</shadows>
</scene>
<wind/>
<spherical_coordinates>
  <surface_model>EARTH_WGS84</surface_model>
  <latitude_deg>0</latitude_deg>
  <longitude_deg>0</longitude_deg>
  <elevation>0</elevation>
  <heading_deg>0</heading_deg>
</spherical_coordinates>
<model name='unit_box'>
  <pose>1.51271 -0.181418 0.5 0 -0 0</pose>
  <link name='link'>
    <inertial>
      <mass>1</mass>
      <inertia>
        <ixx>0.166667</ixx>
        <ixy>0</ixy>
        <ixz>0</ixz>
        <iyy>0.166667</iyy>
        <iyz>0</iyz>
        <izz>0.166667</izz>
      </inertia>
      <pose>0 0 0 -0 0</pose>
    </inertial>
  </link>
</model>

```

```

<collision name='collision'>
  <geometry>
    <box>
      <size>1 1 1</size>
    </box>
  </geometry>
  <max_contacts>10</max_contacts>
  <surface>
    <contact>
      <ode/>
    </contact>
    <bounce/>
    <friction>
      <torsional>
        <ode/>
      </torsional>
      <ode/>
    </friction>
  </surface>
</collision>
<visual name='visual'>
  <geometry>
    <box>
      <size>1 1 1</size>
    </box>
  </geometry>
  <material>
    <script>
      <name>Gazebo/Grey</name>
      <uri>file://media/materials/scripts/gazebo.material</uri>
    </script>
  </material>
</visual>
<self_collide>0</self_collide>
<enable_wind>0</enable_wind>
<kinematic>0</kinematic>
</link>
</model>
<model name='unit_sphere'>
  <pose>-1.89496 2.36764 0.5 0 -0 0</pose>
  <link name='link'>
    <inertial>
      <mass>1</mass>
      <inertia>
        <ixx>0.1</ixx>
        <ixy>0</ixy>
        <ixz>0</ixz>
        <iyy>0.1</iyy>
        <iyz>0</iyz>
        <izz>0.1</izz>
      </inertia>
      <pose>0 0 0 0 -0 0</pose>
    </inertial>
    <collision name='collision'>
      <geometry>
        <sphere>
          <radius>0.5</radius>
        </sphere>
      </geometry>
      <max_contacts>10</max_contacts>
      <surface>
        <contact>
          <ode/>
        </contact>
        <bounce/>
        <friction>
          <torsional>
            <ode/>
          </torsional>
          <ode/>
        </friction>
      </surface>
    </collision>
  </link>
</model>

```

```

<visual name='visual'>
  <geometry>
    <sphere>
      <radius>0.5</radius>
    </sphere>
  </geometry>
  <material>
    <script>
      <name>Gazebo/Grey</name>
      <uri>file://media/materials/scripts/gazebo.material</uri>
    </script>
  </material>
</visual>
<self_collide>0</self_collide>
<enable_wind>0</enable_wind>
<kinematic>0</kinematic>
</link>
</model>
<state world_name='default'>
  <sim_time>0 0</sim_time>
  <real_time>0 0</real_time>
  <wall_time>1626668720 808592627</wall_time>
  <iterations>0</iterations>
  <model name='ground_plane'>
    <pose>0 0 0 0 -0 0</pose>
    <scale>1 1 1</scale>
    <link name='link'>
      <pose>0 0 0 0 -0 0</pose>
      <velocity>0 0 0 0 -0 0</velocity>
      <acceleration>0 0 0 0 -0 0</acceleration>
      <wrench>0 0 0 0 -0 0</wrench>
    </link>
  </model>
  <model name='unit_box'>
    <pose>1.51272 -0.181418 0.499995 0 1e-05 0</pose>
    <scale>1 1 1</scale>
    <link name='link'>
      <pose>1.51272 -0.181418 0.499995 0 1e-05 0</pose>
      <velocity>0 0 0 0 -0 0</velocity>
      <acceleration>0.010615 -0.006191 -9.78231 0.012424 0.021225 1.8e-
05</acceleration>
      <wrench>0.010615 -0.006191 -9.78231 0 -0 0</wrench>
    </link>
  </model>
  <model name='unit_sphere'>
    <pose>-0.725833 1.36206 0.5 0 -0 0</pose>
    <scale>1 1 1</scale>
    <link name='link'>
      <pose>-0.944955 1.09802 0.5 0 -0 0</pose>
      <velocity>0 0 0 0 -0 0</velocity>
      <acceleration>0 0 0 0 -0 0</acceleration>
      <wrench>0 0 0 0 -0 0</wrench>
    </link>
  </model>
  <light name='sun'>
    <pose>0 0 10 0 -0 0</pose>
  </light>
</state>
<gui fullscreen='0'>
  <camera name='user_camera'>
    <pose>3.17226 -5.10401 6.58845 0 0.739643 2.19219</pose>
    <view_controller>orbit</view_controller>
    <projection_type>perspective</projection_type>
  </camera>
</gui>
</world>
</sdf>

```

III. Edit the following in the `gz_sim` node in your launch file:

```
# add the world path
world_file_path = os.path.join(pkg_share, 'world', 'my_world.sdf')
# gz launch
gazebo = IncludeLaunchDescription(
    PythonLaunchDescriptionSource([os.path.join(
        get_package_share_directory('ros_gz_sim'),
        'launch',
        'gz_sim.launch.py'
    )]),
    launch_arguments={'gz_args': f'-r -v 4 {world_file_path}'}.items())
```

IV. Add the *world* folder in the share section in `CMakeLists.txt`

```
install(
  DIRECTORY urdf meshes launch rviz config world
  DESTINATION share/${PROJECT_NAME}
)
```

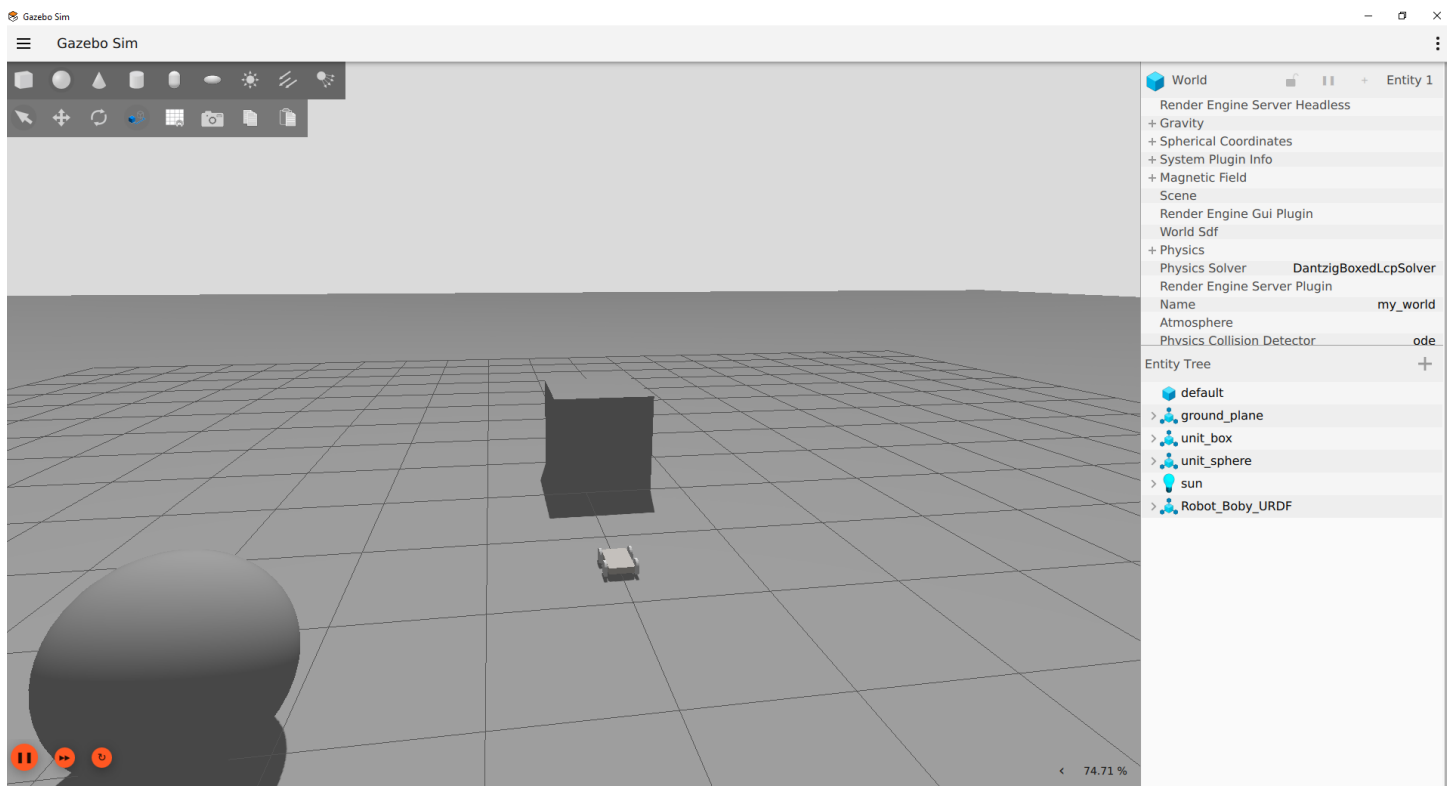
V. Build the workspace, then launch the file

```
cd ~/design_ws
```

```
colcon build --symlink-install --packages-select
mobile_robot_sim
```

```
source install/setup.bash
```

```
ros2 launch mobile_robot_sim robot_gazebo.launch.py
```



➤ Sensors Setup

The last step for robot localization is IMU sensor setup and sensor fusion with odometry.

- I. Add the IMU sensor in the URDF after the last joint in the robot (after `</joint>` and before `<ros2_control>`)

```
<!--Adding the sensor IMU-->
<link name="imu_link">
  <inertial>
    <mass value="0.05"/>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <inertia ixx="1e-5" ixy="0" ixz="0"
      iyy="1e-5" iyz="0"
      izz="1e-5"/>
  </inertial>

  <visual>
    <geometry>
      <box size="0.02 0.02 0.01"/>
    </geometry>
  </visual>

  <collision>
    <geometry>
      <box size="0.02 0.02 0.01"/>
    </geometry>
  </collision>
</link>

<joint name="imu_joint" type="fixed">
  <parent link="RobotBody"/>
  <child link="imu_link"/>
  <origin xyz="0 0 0.05" rpy="0 0 0"/>
</joint>
```

- II. Add the IMU gz plugin at the end before `</robot>`

```
<!--Adding IMU Sensor Plugin-->
<gazebo reference="imu_link">
  <sensor name="imu_sensor" type="imu">
    <always_on>true</always_on>
    <update_rate>50</update_rate>
    <topic>imu</topic>
    <gz_frame_id>imu_link</gz_frame_id>
    <plugin
      filename="gz-sim-imu-system"
      name="gz::sim::systems::Imu"/>
  </sensor>
</gazebo>
```

- III. Update the bridge node for IMU sensor as follows:

```
#Bridge Node
bridge_gz = Node(
    package='ros_gz_bridge',
    executable='parameter_bridge',
    arguments=[
        # General
        '/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock',

        # Gazebo_Control
        '/cmd_vel@geometry_msgs/msg/Twist[gz.msgs.Twist',
        '/odom@nav_msgs/msg/Odometry[gz.msgs.Odometry',
        '/joint_states@sensor_msgs/msg/JointState[gz.msgs.Model',
        '/imu@sensor_msgs/msg/Imu[gz.msgs.IMU',
    ],
    remappings=[
        ('/world/empty/model/Robot_Body_URDF/joint_state',
        'joint_states'),
    ],
    output='screen')
```

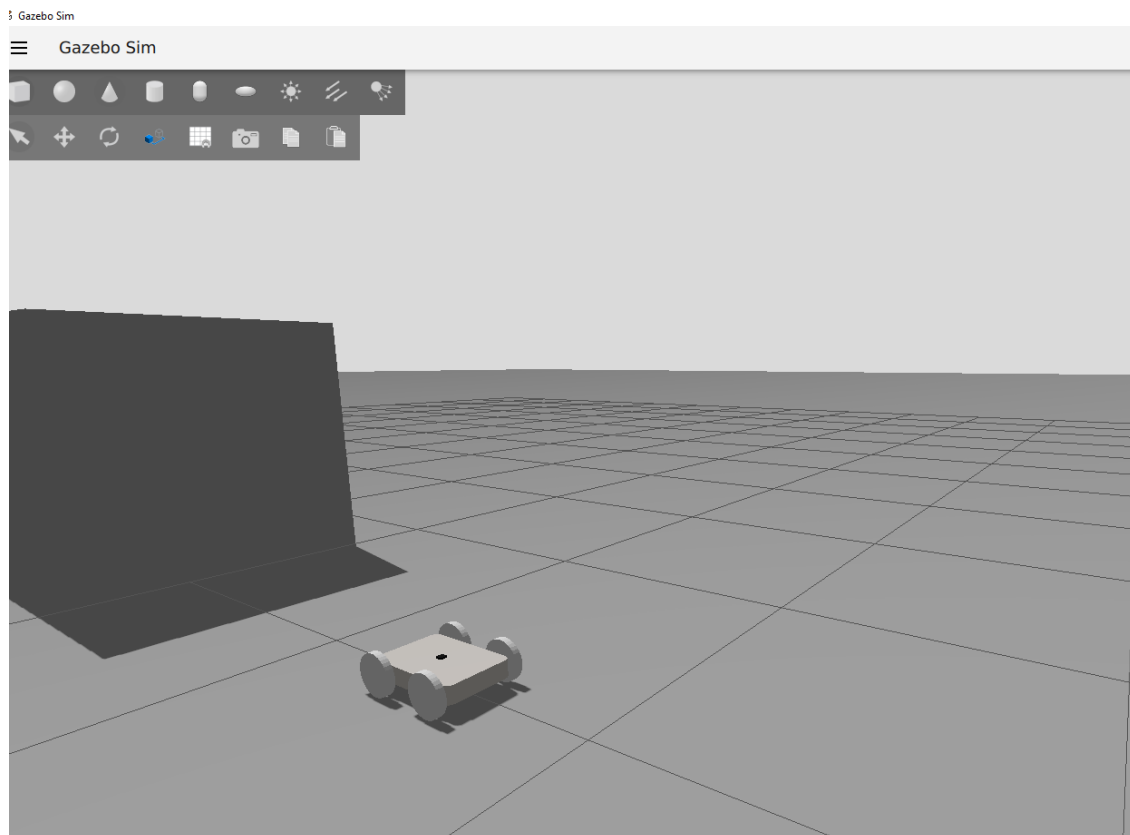
IV. Build workspace, then launch the file.

```
cd ~/design_ws
```

```
colcon build --symlink-install --packages-select
mobile_robot_sim
```

```
source install/setup.bash
```

```
ros2 launch mobile_robot_sim robot_gazebo.launch.py
```



V. You can check for the topics `/imu` and `/diff_drive_controller/odom` for odometry and orientation readings

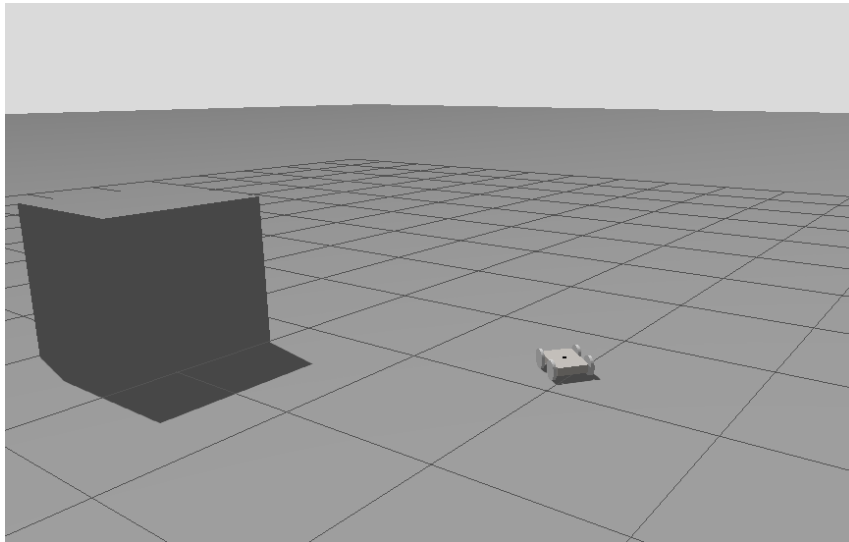
```
dina2997@DESKTOP-E9CU3F0:~$ ros2 topic list
/clock
/cmd_vel
/controller_manager/activity
/controller_manager/introspection_data/full
/controller_manager/introspection_data/names
/controller_manager/introspection_data/values
/controller_manager/statistics/full
/controller_manager/statistics/names
/controller_manager/statistics/values
/diagnostics
/diff_drive_controller/cmd_vel
/diff_drive_controller/odom
/diff_drive_controller/transition_event
/dynamic_joint_states
/imu
/joint_state_broadcaster/transition_event
/joint_states
/odom
/parameter_events
/robot_description
/rosout
/scan
/tf
/tf_static
```

VI. Also, you can check for the position update using publishing velocity to the `/diff_drive_controller/cmd_vel`

```
ros2 topic pub /diff_drive_controller/cmd_vel
geometry_msgs/msg/TwistStamped "{header: {stamp: now, frame_id:
'RobotBody'}, twist: {linear: {x: 0.2, y: 0.0, z: 0.0},
angular: {x: 0.0, y: 0.0, z: 0.5}}}" --rate 5
```

```
dina2997@DESKTOP-E9CU3F0:~$ ros2 topic pub /diff_drive_controller/cmd_vel geometry_msgs/msg/TwistStamped "{header: {stamp: now, frame_id: 'RobotBody'}, twist: {linear: {x: 0.2, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.5}}}" --rate 5
publisher: beginning loop
publishing #1: geometry_msgs.msg.TwistStamped(header=std_msgs.msg.Header(stamp=builtin_interfaces.msg.Time(sec=1771521537, nanosec=450149459), frame_id='RobotBody'), twist=geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.2, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.5)))
```

You can check for the position update and the movement of the robot



```
dina2997@DESKTOP-E9CU3FO:~/design_ws$ ros2 topic echo /diff_drive_controller/odom --once
header:
  stamp:
    sec: 1
    nanosec: 540000000
  frame_id: ''
  child_frame_id: ''
  pose:
    pose:
      position:
        x: -1.9494107282790858e-11
        y: -2.8873563546402738e-22
        z: 0.0
      orientation:
        x: 0.0
        y: 0.0
        z: 1.0734462137260981e-11
        w: 1.0
```

➤ *Obstacle Avoidance Setup*

Let's set up the sensor for obstacle avoidance and create the node to check for the obstacles and make the robot avoids any.

- I. Add the ultrasonic sensor link in the robot and attach to base of the robot in the URDF. You can add after the IMU joint.

```

<link name="ultrasonic_link">
  <visual>
    <geometry>
      <cylinder radius="0.03" length="0.05"/>
    </geometry>
  </visual>
  <collision>
    <geometry>
      <cylinder radius="0.03" length="0.05"/>
    </geometry>
  </collision>

  <inertial>
    <mass value="0.1"/>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <inertia ixx="1e-5" ixy="0" ixz="0"
      iyy="1e-5" iyz="0"
      izz="1e-5"/>
  </inertial>
</link>
<joint name="ultrasonic_joint" type="fixed">
  <parent link="RobotBody"/>
  <child link="ultrasonic_link"/>
  <origin xyz="0.15 0 0.07" rpy="0 0 0"/>
</joint>

```

II. Add the ray plugin for the ultrasonic sensor for gz sim. You can add it before **</robot>**

```

<gazebo reference="ultrasonic_link">
  <sensor name="lidar_sensor" type="gpu_lidar">
    <update_rate>50</update_rate>
    <topic>scan</topic>

    <ray>
      <scan>
        <horizontal>
          <samples>640</samples>
          <resolution>1</resolution>
          <min_angle>0</min_angle>
          <max_angle>0</max_angle>
        </horizontal>
        <vertical>
          <samples>1</samples>
          <resolution>0.01</resolution>
          <min_angle>0</min_angle>
          <max_angle>0</max_angle>
        </vertical>
      </scan>
      <range>
        <min>0.08</min>
        <max>10.0</max>
        <resolution>0.01</resolution>
      </range>
    </ray>
    <always_on>1</always_on>
    <visualize>true</visualize>
  </sensor>
</gazebo>

```

III. Update the bridge node for Laser sensor topic as follows:

```
#Bridge Node
bridge_gz = Node(
    package='ros_gz_bridge',
    executable='parameter_bridge',
    arguments=[
        # General
        '/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock',

        # Gazebo_Control
        '/cmd_vel@geometry_msgs/msg/Twist[gz.msgs.Twist',
        '/odom@nav_msgs/msg/Odometry[gz.msgs.Odometry',
        '/joint_states@sensor_msgs/msg/JointState[gz.msgs.Model',
        '/imu@sensor_msgs/msg/Imu[gz.msgs.IMU',
        '/scan@sensor_msgs/msg/LaserScan[gz.msgs.LaserScan',
    ],
    remappings=[
        ('/world/empty/model/Robot_Body_URDF/joint_state',
        'joint_states'),
    ],
    output='screen')
```

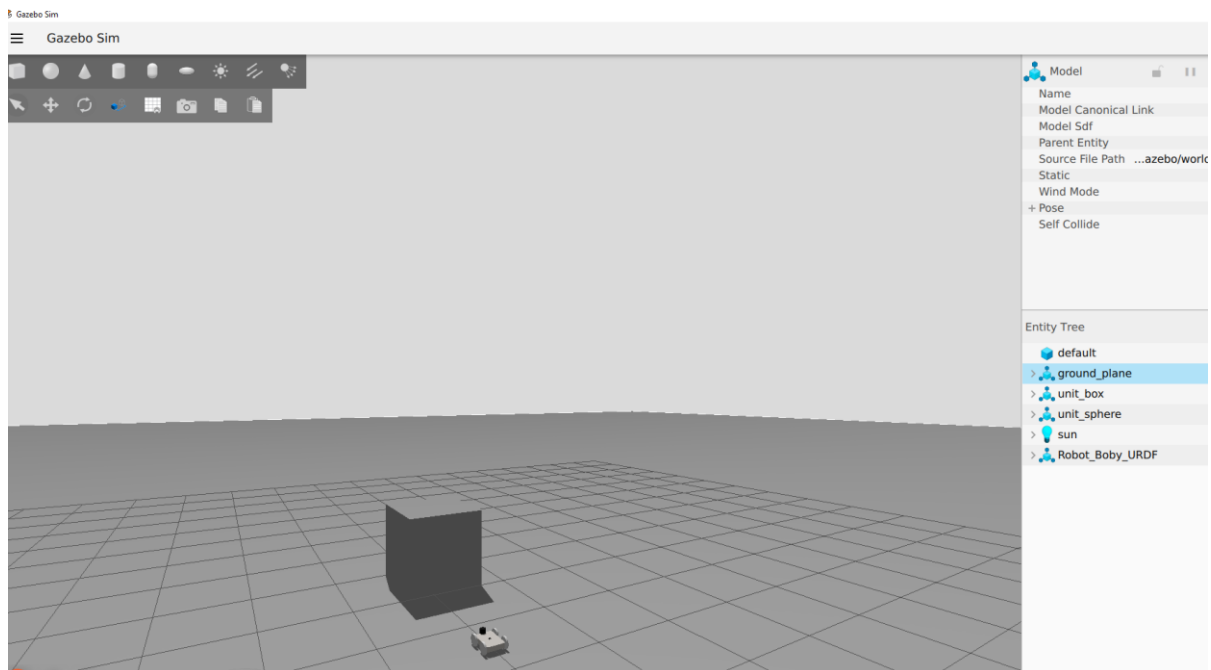
IV. Build workspace, then launch the file.

```
cd ~/design_ws
```

```
colcon build --symlink-install --packages-select
mobile_robot_sim
```

```
source install/setup.bash
```

```
ros2 launch mobile_robot_sim robot_gazebo.Launch.py
```



You can check `/scan` topic is published

```
dina2997@DESKTOP-E9CU3F0:~$ ros2 topic list
/clock
/cmd_vel
/controller_manager/activity
/controller_manager/introspection_data/full
/controller_manager/introspection_data/names
/controller_manager/introspection_data/values
/controller_manager/statistics/full
/controller_manager/statistics/names
/controller_manager/statistics/values
/diagnostics
/diff_drive_controller/cmd_vel
/diff_drive_controller/odom
/diff_drive_controller/transition_event
/dynamic_joint_states
/imu
/joint_state_broadcaster/transition_event
/joint_states
/odom
/odometry/filtered
/parameter_events
/robot_description
/rosout
/scan
/set_pose
/tf
/tf_static
```